

Processo Unificado

PROCESSO DE SOFTWARE — Professor: Me. Sergio Souza Novak —
Faculdade de Engenharia de Software



1 Modelo de Processo Unificado

2 Modelo Cascata

3 Comparação entre Modelos

4 Dualidade Produto e Processo

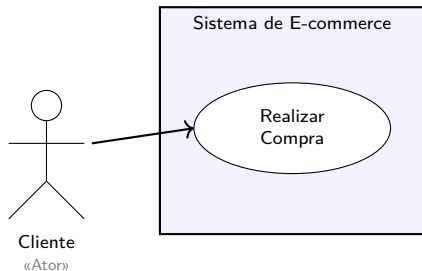
- **Fusão de Modelos:** Tentativa de aproveitar os melhores recursos dos modelos tradicionais de processo de software.
- **Princípios Ágeis:** Implementa muitos dos melhores princípios do desenvolvimento ágil de software.
- **Foco no Cliente:** Reconhece a importância da comunicação contínua e efetiva com o cliente.
- **Casos de Uso:** Utiliza métodos racionalizados (casos de uso) para descrever a visão do cliente sobre o sistema.

- **Papel da Arquitetura:** Enfatiza fortemente o papel central da arquitetura de software no desenvolvimento.
- **Metas Arquiteturais:** Ajuda o arquiteto a focar em:
 - Compreensibilidade do sistema
 - Confiança em mudanças futuras
 - Reutilização de componentes
- **Fluxo Iterativo e Incremental:** Sugere um fluxo de processo que proporciona uma evolução contínua.
- **Sensação Evolucionária:** Essencial no contexto do desenvolvimento de software moderno.

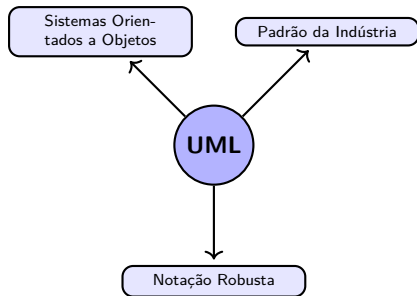
O que é um Caso de Uso?

- Descreve a interação entre o sistema e o usuário (ou outro sistema).
- Define **o que** o sistema deve fazer (requisitos), a partir da perspectiva do usuário.
- Representa um resultado de valor para um **Ator**.

Exemplo: Um *Cliente* (Ator) interagindo com um Sistema de E-commerce para realizar uma compra.

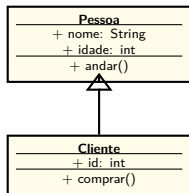


- **Linguagem de Modelagem Unificada:** Desenvolvida para apoiar o trabalho de engenharia de software.
- **Notação Robusta:** Focada na modelagem e desenvolvimento de sistemas orientados a objetos.
- **Padrão da Indústria:** Utilizada globalmente para o desenvolvimento de software de todos os tipos.

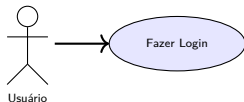


Exemplos de Diagramas UML

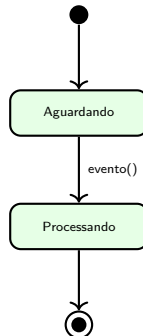
1. Diagrama de Classes



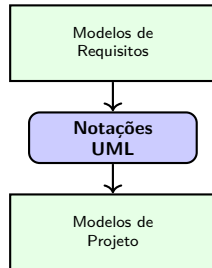
2. Diagrama de Casos de Uso



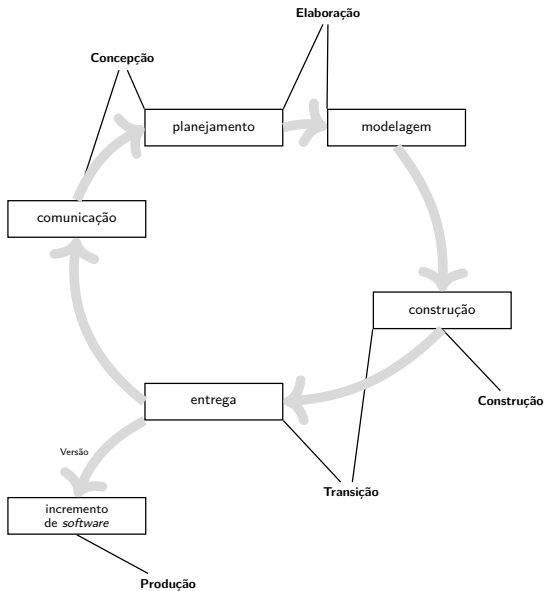
3. Diagrama de Estados

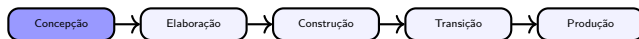


- **Modelos de Requisitos:** Utilizada para entender e documentar o que o sistema deve fazer.
- **Modelos de Projeto:** Utilizada para especificar como o sistema será construído.
- **Aprendizado:** Conhecer as regras básicas de notação e modelagem UML é fundamental para desenvolvedores.

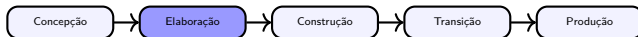


Fases do Processo Unificado

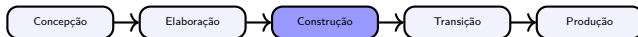




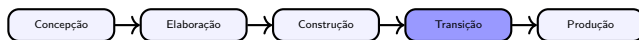
- **Foco:** Comunicação com o cliente e planejamento.
- **Requisitos:** Descritos em um conjunto de casos de uso preliminares.
- **Arquitetura:** Define recursos e funções esperados por cada categoria de usuário.
- **Planejamento:** Identifica recursos, avalia os principais riscos e define um cronograma preliminar para os incrementos.



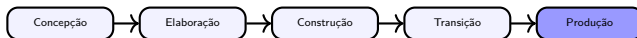
- **Foco:** Atividades de planejamento e modelagem.
- **Ações:** Refina e expande os casos de uso preliminares.
- **Arquitetura Base:** Criação considerando cinco visões: modelo de caso de uso, análise, projeto, implementação e entrega.
- **Ajustes:** Normalmente, as modificações no planejamento preliminar são feitas nesta fase.



- **Foco:** Idêntica à atividade de construção do processo genérico.
- **Implementação:** Código-fonte para todos os recursos do incremento de software.
- **Testes e Integração:** Testes de unidade para componentes, montagem e testes de integração.
- **Aceitação:** Casos de uso formam pacotes de testes executados antes da próxima fase.



- **Foco:** Últimos estágios de construção e primeira parte de emprego genérico (entrega e feedback).
- **Testes Beta:** Software entregue aos usuários para avaliação.
- **Feedback:** Relata defeitos e mudanças necessárias.
- **Conclusão:** O incremento torna-se uma versão utilizável do software.



- **Produção:** Coincide com a atividade de entrega genérica.
 - Monitora o uso contínuo, provê suporte, gere relatórios de defeitos e mudanças.
- **Concomitância:** Fases podem ocorrer de forma simultânea e escalonada.
- **Adaptação:** A equipe adapta o processo (ações, tarefas, artefatos) às suas necessidades; nem toda tarefa é conduzida em todos os projetos.

- **Requisitos Compreendidos:** Quando os requisitos de um problema são bem definidos e razoavelmente estáveis.
- **Fluxo Linear:** O trabalho flui da comunicação à entrega de modo linear.
- **Aperfeiçoamentos:** Comum em adaptações bem-definidas de sistemas existentes (ex.: adaptação em software contábil devido a novas normas).
- **Projetos Novos:** Pode ocorrer em novos esforços de desenvolvimento, de forma restrita.

Modelo Cascata: O que é?

- Também chamado de **modelo sequencial linear**.
- Sugere uma abordagem sequencial e sistemática para o desenvolvimento de software.
- **Fases principais:**
 - 1 Especificação dos requisitos do cliente.
 - 2 Planejamento.
 - 3 Modelagem.
 - 4 Construção.
 - 5 Entrega e Suporte contínuo do software concluído.

- **Histórico:** É o paradigma mais antigo da engenharia de software (mais de cinco décadas).
- **Eficácia Questionada:** Ao longo do tempo, as críticas a esse modelo de processo cresceram.
- **Mudança de Cenário:** Devido às características dinâmicas dos projetos modernos, até mesmo seus defensores mais áduos questionam sua eficácia nos dias atuais.

Comparação entre Modelos de Processo (Parte 1)

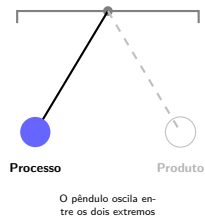
Modelo	Prós	Contras
Cascata	● Fácil de entender e planejar ● Funciona para projetos pequenos e bem compreendidos ● Análise e teste simples e diretos	● Não se adapta bem a mudanças ● Teste ocorre nas fases finais ● Aprovação do cliente vem no final
Prototipação	● Impacto de alterações nos requisitos é reduzido ● Cliente se envolve desde o início ● Funciona bem para projetos pequenos ● Probabilidade de rejeição reduzida	● Envolvimento do cliente pode causar atrasos ● Tentação de “embalar” o protótipo ● Desperdício de trabalho em protótipo descartável ● Difícil de planejar e gerenciar

Comparação entre Modelos de Processo (Parte 2)

Modelo	Prós	Contras
Espiral	● Envolvimento contínuo dos clientes ● Riscos de desenvolvimento gerenciados ● Adequado para projetos grandes e complexos ● Funciona bem para artefatos extensíveis	● Falhas na análise de risco podem comprometer o projeto ● Projeto pode ser difícil de gerenciar ● Exige equipe especializada
Processo Unificado	● Documentação de alta qualidade enfatizada ● Envolvimento contínuo dos clientes ● Adapta-se a alterações nos requisitos ● Funciona bem para projetos de manutenção	● Casos de uso nem sempre são precisos ● Integração de incrementos é complicada ● Sobreposição de fases pode causar problemas ● Exige equipe especializada

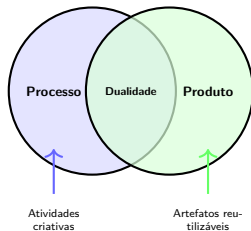
Processo Fraco vs. Confiança Excessiva

- Se o **processo for fraco**, o produto final sofrerá consequências.
- Uma **confiança excessiva e obsessiva** no processo é igualmente perigosa.
- Margaret Davis [Dav95a]: a cada ~ 10 anos, a comunidade de software redefine “o problema”, mudando o foco de itens do *produto* para itens do *processo*.



A Dualidade Produto e Processo

- As oscilações entre foco em produto e processo **não resolvem “o problema”**.
- Produto e processo devem ser tratados como uma **dualidade** (coexistência), não como uma **dicotomia** (oposição).
- Não se pode compreender o artefato completo olhando *apenas* como processo ou *apenas* como produto.
- A reutilização de produtos aumenta a satisfação e reforça a urgência de aceitar essa dualidade.



- As pessoas obtêm satisfação tanto do **processo criativo** quanto do **produto final**.
- **Analogias:**
 - Um *artista* sente prazer com suas pinceladas e com o quadro finalizado.
 - Um *escritor* sente prazer com a busca da metáfora e com o livro concluído.
- Como **profissional de software criativo**, deve-se extrair satisfação do processo e do produto final.
- A **dualidade produto e processo** é um elemento importante para manter pessoas criativas engajadas à medida que a engenharia de software evolui.